Manual técnico — TutoríasGT

1. Introducción

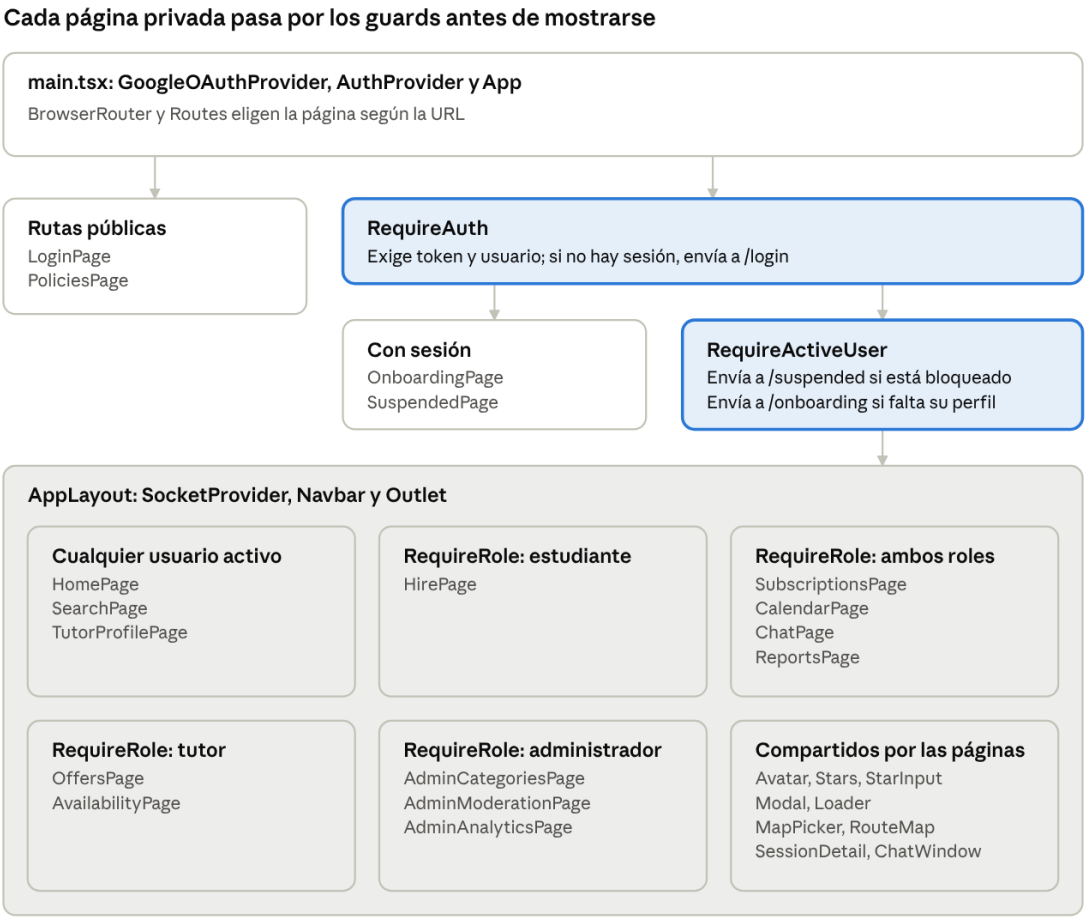
TutoríasGT es un marketplace educativo que conecta estudiantes con tutores y automatiza la contratación de clases virtuales y presenciales. Incluye cálculo de traslado con mapas, prevención de traslapes de horario, suscripciones por bloques, chat en tiempo real y moderación de cuentas.

El sistema tiene tres piezas: un cliente React, una API REST con WebSockets y una base de datos PostgreSQL. El cliente se publica en Netlify y la API y la base de datos en Render.

Capa	Tecnología	Uso en el proyecto
Frontend	React 19, Vite 6	Interfaz de una sola página (SPA)
Lenguaje	TypeScript 5	Tipado estricto en frontend y backend
Estilos	SASS/SCSS	Variables, mixins y CSS Modules
Rutas	React Router 7	Navegación y protección por rol
Peticiones	Axios	Cliente HTTP con JWT automático
Mapas	Leaflet, OpenStreetMap, OSRM	Selección de ubicación y ruta real por calles
Calendario	react-big-calendar, date-fns	Vistas diaria, semanal y mensual
Gráficas	Recharts	Reportes del administrador
Tiempo real	Socket.io	Chat y confirmación de lectura
Backend	Node.js 20+, Express 5	API REST modular
Validación	Zod	Validación de cada petición
Base de datos	PostgreSQL, Prisma 6	Modelo de datos y migraciones
Autenticación	Google OAuth 2.0, JWT	Inicio de sesión y sesión del usuario

2. Esquema conceptual de componentes

La aplicación es un árbol de componentes con rutas anidadas: cada nivel agrega una condición de acceso y entrega su contenido al siguiente por medio de `Outlet`.



Esquema conceptual de componentes y rutas

Los recuadros resaltados son guards. Las páginas dentro de AppLayout comparten la barra de navegación y la conexión del chat.

3. Descripción de componentes

El frontend separa páginas, componentes reutilizables, contextos y funciones de API. Cada página obtiene sus datos por medio de `src/api/` y nunca escribe URLs directamente.

Páginas

Página	Ruta	Rol	Función
LoginPage	`/login`	Pública	Inicio de sesión con Google
PoliciesPage	`/policies`	Pública	Políticas de convivencia
OnboardingPage	`/onboarding`	Con sesión	Perfil extendido y selección de rol

Página	Ruta	Rol	Función
SuspendedPage	`/suspended`	Con sesión	Motivo del bloqueo y apelación
HomePage	`/`	Activo	Bienvenida y accesos directos
SearchPage	`/search`	Activo	Búsqueda de tutores con filtros
TutorProfilePage	`/tutors/:id`	Activo	Perfil, ofertas, reseñas y WhatsApp
HirePage	`/hire/:tutorId/:offerId`	Estudiante	Presupuesto, ruta y contratación
SubscriptionsPage	`/subscriptions`	Estudiante, tutor	Contrataciones y solicitudes
CalendarPage	`/calendar`	Estudiante, tutor	Vistas día, semana y mes
ChatPage	`/chat/:conversationId?`	Estudiante, tutor	Mensajería en tiempo real
ReportsPage	`/reports`	Estudiante, tutor	Crear y consultar denuncias
OffersPage	`/tutor/offers`	Tutor	CRUD de ofertas
AvailabilityPage	`/tutor/availability`	Tutor	Horario semanal
AdminCategoriesPage	`/admin/categories`	Admin	CRUD de categorías
AdminModerationPage	`/admin/moderation`	Admin	Denuncias, apelaciones, tutores y usuarios
AdminAnalyticsPage	`/admin/analytics`	Admin	Ranking, demanda y estadísticas

Componentes reutilizables

Componente	Ubicación	Función
AppLayout	`components/layout`	Barra superior, contenido y conexión del socket
Navbar	`components/layout`	Menú por rol con versión móvil
Avatar	`components/ui`	Foto del usuario o su inicial
Stars	`components/ui`	Muestra una calificación
StarInput	`components/ui`	Permite elegir de 1 a 5 estrellas

Componente	Ubicación	Función
Modal	`components/ui`	Ventana emergente; cierra con Escape o clic afuera
Loader	`components/ui`	Indicador de carga
MapPicker	`components/map`	Marca una ubicación con clic, arrastre o geolocalización
RouteMap	`components/map`	Dibuja la ruta entre tutor y estudiante
SessionDetail	`components/calendar`	Detalle, acciones y reseña de una sesión
ChatWindow	`components/chat`	Conversación abierta, envío y lectura
ReportsTab, AppealsTab, PendingTutorsTab, UsersTab	`components/admin`	Pestañas de moderación
UserHistoryModal	`components/admin`	Historial de denuncias y apelaciones por usuario

Contextos, guards y API

- AuthContext guarda el usuario y el token, cierra sesión cuando el JWT expira y ofrece `useAuth()`.
- SocketContext mantiene una sola conexión de Socket.io autenticada con el JWT y ofrece `useSocket()`.
- Guards (`routes/guards.tsx`): `RequireAuth` exige sesión, `RequireActiveUser` envía a suspensión u onboarding y `RequireRole` limita por rol.
- api/client.ts agrega el encabezado `Authorization: Bearer <token>` a cada petición. Ante un 401 borra el token y redirige a `/login`.
- api/*.ts agrupa las llamadas por módulo: usuarios, tutores, ofertas, suscripciones, sesiones, chat, denuncias, apelaciones y administración.

4. Uso de TypeScript

Todo el código está escrito en TypeScript, de modo que los errores de tipos se detectan antes de ejecutar. `npm run build` corre `tsc -b` primero y el despliegue falla si hay errores.

Tipos de la API. `types/api.ts` describe cada respuesta del backend. Los *union types* limitan los valores posibles.

```
export type Role = 'STUDENT' | 'TUTOR' | 'ADMIN';
export type Modality = 'VIRTUAL' | 'TUTOR_PLACE' | 'STUDENT_HOME';
```

Herencia y tipos derivados. `extends` reutiliza una interfaz. `Pick` toma solo algunos campos y `&` combina tipos.

```
export interface SearchResult extends Offer {
  tutor: UserSummary & {
    tutorProfile: Pick<TutorProfile, 'avgRating' | 'reviewCount' | 'bio'> | null;
  };
  distanceKm: number | null;
}
```
Respuestas tipadas. El genérico de Axios indica la forma de los datos que llegan.

```
const { data } = await api.get<SearchResult[]>('/tutors/search', { params: filters
});
```
`Record` exhaustivo. Obliga a definir una entrada por cada valor. Si se agrega una modalidad, el compilador marca este objeto hasta completarlo.

```
export const MODALITY_LABEL: Record<Modality, string> = {
  VIRTUAL: 'Virtual',
  TUTOR_PLACE: 'En casa del tutor',
  STUDENT_HOME: 'A domicilio',
};
```
Funciones genéricas. En los formularios, `field` solo acepta nombres válidos y `value` debe tener el tipo de ese campo.

```
const update = <K extends keyof FormState>(field: K, value: FormState[K]) => {
  setForm((prev) => ({ ...prev, [field]: value }));
};
```
Eventos de Socket.io. `types/socket.ts` define los eventos, así que un nombre mal escrito es un error de compilación.

```
export type ChatSocket = Socket<ServerToClientEvents, ClientToServerEvents>;
```
Backend. Las entradas se validan con esquemas Zod, y `z.infer` genera el tipo a partir del esquema. Prisma genera los tipos de cada tabla.

5. Uso de SASS/SCSS

Los estilos combinan archivos globales en `src/styles/` con un `.module.scss` por componente o página. CSS Modules aplica cada clase solo a su componente, por lo que los nombres no chocan.

Archivo	Contenido
`_variables.scss`	Colores, espaciado, sombras, `$breakpoints` y `$status-colors`
`_mixins.scss`	`up()`, `card` y `flex-center`
`_base.scss`	Reset y clases globales: `.btn`, `.card`, `.badge`, `.field`, `.tag`
`main.scss`	Punto de entrada de los estilos globales

Variables y mixins. Los mapas de SASS centralizan los tamaños de pantalla. El mixin `up()` genera cada media query, y la interfaz se diseña primero para móvil.

```
@mixin up($bp) {
  @media (min-width: map.get($breakpoints, $bp)) {
    @content;
  }
}
```

```
.results {
  grid-template-columns: 1fr;

  @include up(sm) { grid-template-columns: repeat(2, 1fr); }
  @include up(xl) { grid-template-columns: repeat(3, 1fr); }
}
```

Mapas y `@each`. Una clase de estado se genera por cada entrada del mapa. Agregar un estado solo requiere una línea en `$status-colors`.

```
.badge {
  @each $status, $color in $status-colors {
    &--#{$status} { background: $color; }
  }
}
```

Anidación y operadores. `&` referencia al selector actual, y los espacios se calculan a partir de `$space`.

```
.btn {
  padding: $space $space * 2;

  &:hover { background: color.adjust($color-primary, $lightness: -8%); }
  &--outline { background: transparent; }
}
```

Funciones de color. `color.adjust` oscurece un color al pasar el mouse, y `rgba($color-primary, 0.08)` crea tintes de la misma paleta.

Estilos de librerías. `:global` permite adaptar clases externas sin que CSS Modules las renombre, como en el calendario:

```
.calendar {
  :global {
    .rbc-toolbar button.rbc-active { background: $color-primary; }
  }
}
```

6. Diagrama entidad-relación

La base de datos PostgreSQL tiene 14 tablas, definidas en `backend/prisma/schema.prisma`. La tabla User se relaciona con casi todas las demás.

User es el centro del modelo: casi todas las tablas dependen de él

Cardinalidad junto a cada línea. 2x = dos relaciones (por ejemplo, estudiante y tutor).

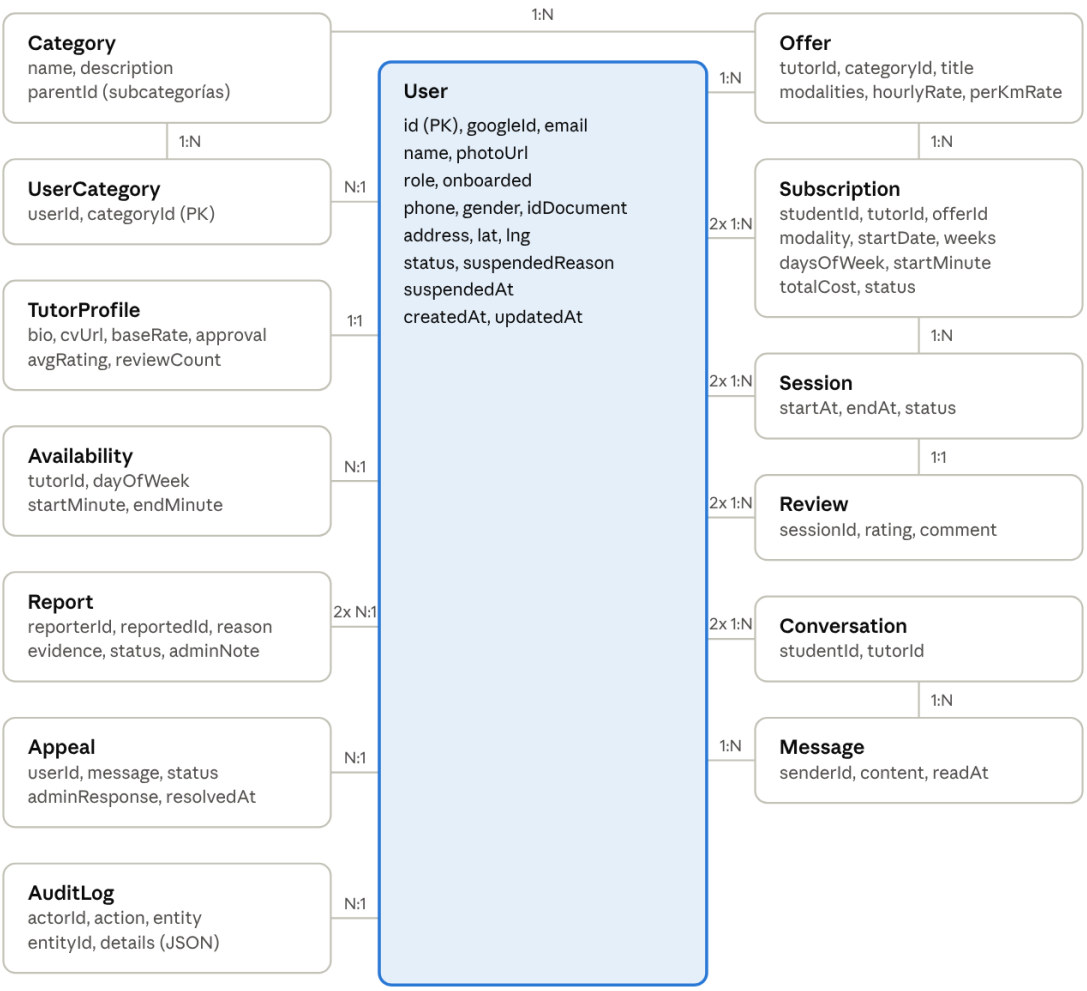


Diagrama entidad-relación · 14 tablas

Tabla	Propósito
User	Cuenta, rol, datos de perfil y estado de suspensión
TutorProfile	Bio, hoja de vida, tarifa base, aprobación y promedio de calificación
Category	Categorías de dos niveles mediante `parentId`
UserCategory	Afinidad entre usuarios y categorías
Offer	Tutoría publicada, con modalidades y tarifas
Availability	Bloques semanales del tutor, en minutos desde medianoche
Subscription	Contratación de un plan con costos calculados

Tabla	Propósito
Session	Cada clase generada por una suscripción
Review	Calificación de una sesión; una por sesión
Conversation	Canal único entre un estudiante y un tutor
Message	Mensaje con marca de lectura `readAt`
Report	Denuncia entre usuarios
Appeal	Apelación de una cuenta suspendida
AuditLog	Registro de acciones relevantes

7. Backend

La API sigue una estructura modular: cada módulo tiene `routes` (rutas y permisos), `controller` (entrada y salida HTTP) y `service` (reglas de negocio). Todas las rutas se publican bajo `/api`.

Módulo	Ruta base	Responsabilidad
auth	`/auth`	Login con Google, emisión de JWT, usuario actual
users	`/users`	Onboarding y perfil
categories	`/categories`	Consulta pública para usuarios y CRUD para admin
tutors	`/tutors`	Búsqueda, perfil y reseñas
offers	`/offers`	Ofertas del tutor
availability	`/availability`	Horario semanal
quotes	`/quotes`	Presupuesto y ruta
subscriptions	`/subscriptions`	Contratación y aprobación
sessions	`/sessions`	Sesiones, cumplimiento y cancelación
reviews	`/reviews`	Calificaciones
conversations	`/conversations`	Historial de chat
reports	`/reports`	Denuncias
appeals	`/appeals`	Apelaciones
admin	`/admin`	Moderación, auditoría y analítica

Autenticación y JWT. El cliente envía la credencial de Google a `POST /auth/google`. El backend la verifica, crea o actualiza el usuario y emite un JWT cuya duración se define en `JWT_EXPIRES_IN`.

Cada ruta protegida valida el encabezado Bearer con `requireAuth`. Si el token falta o expiró, responde 401 y el cliente cierra la sesión.

Cuentas suspendidas. `blockSuspended` responde 403 con el código `ACCOUNT_SUSPENDED`. Solo `/auth/me` y `/appeals` siguen disponibles para el usuario bloqueado.

Cálculo de traslado. `quotes.service` consulta OSRM para obtener la distancia real por calles y la geometría de la ruta. Si OSRM falla, usa la distancia en línea recta multiplicada por 1.3.

El traslado por sesión es la distancia de ida y vuelta por la tarifa por kilómetro, y solo se cobra en modalidad a domicilio.

Prevención de traslapes. Al contratar, `subscriptions.service` valida la disponibilidad del tutor y genera todas las sesiones. Luego busca choques con sesiones pendientes o aprobadas, tanto del tutor como del estudiante.

Esa revisión y la creación corren en una transacción `Serializable` con reintentos. Dos solicitudes simultáneas no pueden reservar el mismo horario; la que choca recibe 409 `SCHEDULE_CONFLICT`.

Chat en tiempo real. Socket.io valida el JWT durante la conexión y rechaza cuentas suspendidas. Cada conversación es una sala `conv:<id>`.

Evento	Dirección	Función
`conversation:join`	Cliente a servidor	Entrar a la sala
`conversation:leave`	Cliente a servidor	Salir de la sala
`message:send`	Cliente a servidor	Guardar y reenviar un mensaje
`message:read`	Ambos sentidos	Marcar y avisar lectura
`message:new`	Servidor a cliente	Entregar un mensaje
`conversation:updated`	Servidor a cliente	Actualizar la lista del destinatario

Reseñas y auditoría. Crear una reseña recalcula el promedio del tutor en la misma transacción. Las acciones de moderación, contratación y reseñas quedan registradas en `AuditLog`.

8. Instalación y despliegue

El proyecto requiere Node.js 20 o superior y se ejecuta en dos procesos: `backend` en el puerto 4000 y `frontend` en el 5173.

Instalación local

1. Clonar el repositorio y crear `backend/.env` y `frontend/.env` a partir de `.env.example`.
2. En `backend`, ejecutar `npm install`, `npx prisma generate` y `npm run dev`.
3. En `frontend`, ejecutar `npm install` y `npm run dev`.

Variables de entorno

Variable	Proyecto	Descripción
`DATABASE_URL`	Backend	Conexión a PostgreSQL
`JWT_SECRET`	Backend	Clave para firmar tokens
`JWT_EXPIRES_IN`	Backend	Duración del token, por ejemplo `8h`
`GOOGLE_CLIENT_ID`	Backend	Cliente OAuth de Google
`FRONTEND_URL`	Backend	Orígenes permitidos, separados por coma
`ADMIN_EMAILS`	Backend	Correos con rol de administrador
`TIMEZONE_OFFSET`	Backend	Zona horaria, `-06:00` en Guatemala
`ENABLE_DEV_LOGIN`	Backend	Activa el login de desarrollo
`ALLOW_EARLY_COMPLETE`	Backend	Permite completar sesiones antes de su fin
`VITE_API_URL`	Frontend	URL de la API
`VITE_SOCKET_URL`	Frontend	URL de Socket.io
`VITE_GOOGLE_CLIENT_ID`	Frontend	Cliente OAuth de Google

Despliegue

- Backend y base de datos: Render. Comando de compilación `npm run build` y de inicio `npm start`. La API está en `https://tutorias-api-f9jj.onrender.com/api`.
- Frontend: Netlify, con directorio base `frontend`, comando `npm run build` y carpeta publicada `dist`. `netlify.toml` y `public/_redirects` envían todas las rutas a `index.html`.
- Google OAuth: la URL de Netlify debe estar en los orígenes autorizados de JavaScript.
- CORS: la URL de Netlify debe estar en `FRONTEND_URL`.

Enlace del sitio: pendiente de agregar cuando esté publicado.